

Optimización de búsqueda de números primos

Introducción

El código original calcula números primos entre 1 y 100 000 comprobando divisores desde 2 hasta $n-1$, lo que provoca complejidad $O(n^2)$ y tiempos elevados.

Optimización aplicada

- Reducir comprobaciones hasta $\sqrt{n}$.
- Usar list comprehensions para construir listas.
- Implementar la Criba de Eratóstenes con NumPy para operaciones vectorizadas.

¿Qué es la Criba de Eratóstenes y cómo funciona?

La Criba de Eratóstenes es un método eficiente para encontrar todos los primos hasta un límite N :

1. Crear una lista booleana `sieve[0..N]` inicializada a True (ignorando 0 y 1).
2. Empezar en $p = 2$ (primer primo). Si `sieve[p]` es True, marcar como no primos todos los múltiplos de p desde pp hasta N (pp , $p*p+p$, ...).
3. Incrementar p al siguiente índice True y repetir hasta $p > \sqrt{N}$.
4. Los índices que quedan True son números primos.

Ventajas: - Evita comprobaciones repetidas por número. - Con operaciones vectorizadas (NumPy) las asignaciones de rangos son muy rápidas.

Resultados

Profiling (cProfile)

- Archivo: `profiling_optimizado.txt`
- Función dominante: `primos_optimizado_numpy`

Resultados de profiling

ncalls	tottime	percall	cumtime	percall	función
1	0.000	0.000	0.008	0.008	c:/Users/JM/Documents/GitHu
1	0.008	0.008	0.008	0.008	c:/Users/JM/Documents/GitHu
1	0.000	0.000	0.000	0.000	{built-in method numpy.array}
1	0.000	0.000	0.000	0.000	.../site- packages/numpy/_core/numeric
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

ncalls	totttime	percall	cumtime	percall	función
1	0.000	0.000	0.000	0.000	{built-in method numpy.empty}
1	0.000	0.000	0.000	0.000	.../multiarray.py:1106(copyto)

- tottime: tiempo exclusivo gastado en la función.
- cumtime: tiempo acumulado incluyendo llamadas internas.
- Observación: la función `primos_optimizado_numpy` concentra la mayor parte del tiempo (0.008 s), por lo que es el principal objetivo si se desea optimizar más.

Gráficos

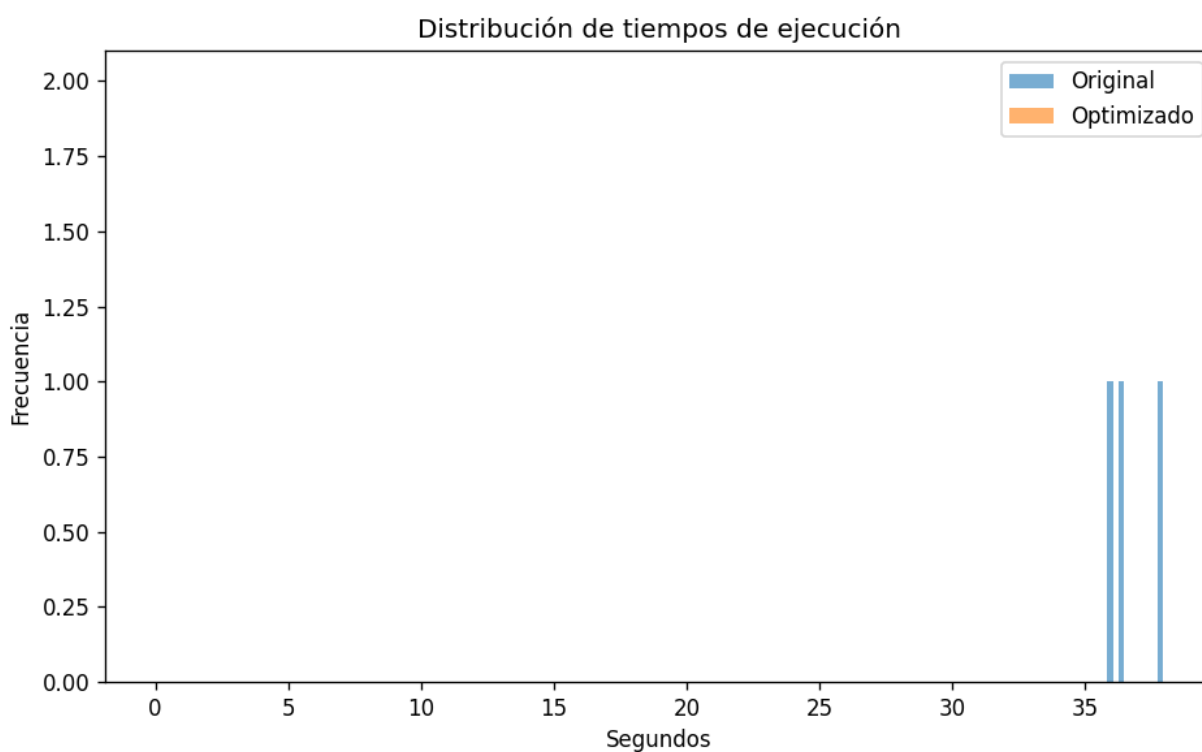


Figure 1: Distribución de tiempos

Conclusiones

La implementación con criba y NumPy reduce de manera drástica el tiempo, dado que no se itera uno a uno los números hasta encontrar los primos, sino que al contar con funciones nativas de NumPy, que se corren en “C” hacen que el proceso sea más eficiente y el gasto computacional sea mínimo.

Enlace al repositorio

Repositorio: [\[ENLACE AL REPOSITORIO\]](#)

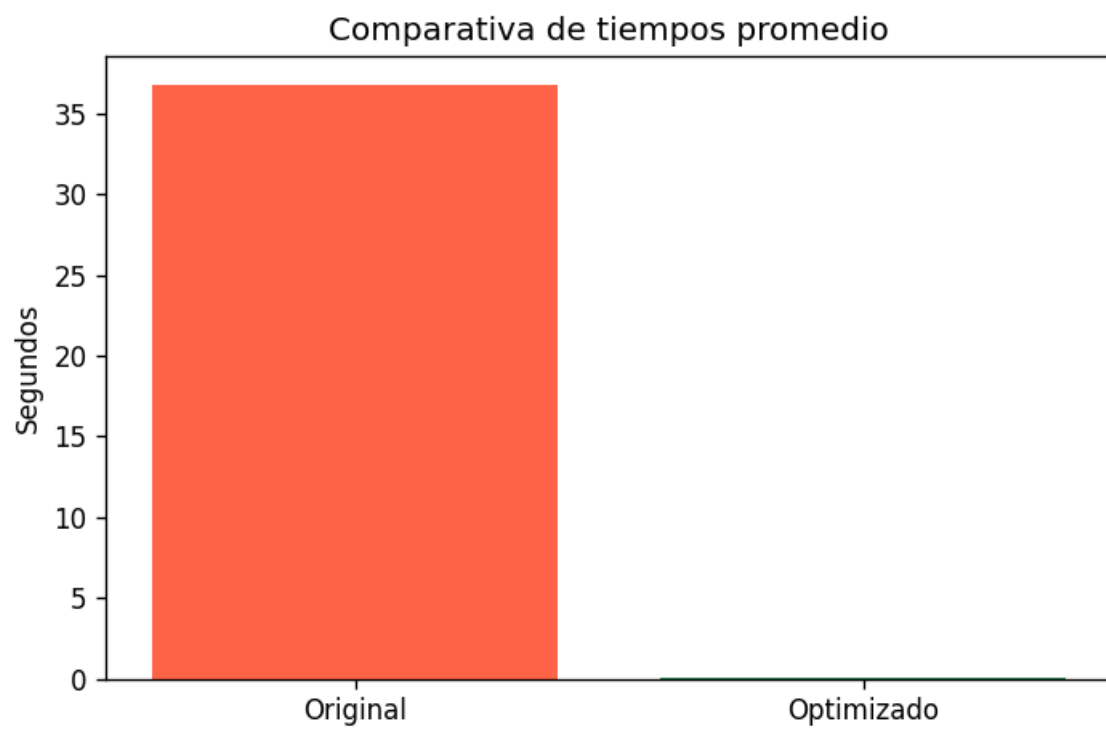


Figure 2: Comparativa de tiempos